

MISSING CHILD IDENTIFICATION USING CONVOLUTIONAL NEURAL NETWORK

**A Major Project Report Submitted
In partial fulfillment of the requirements for the award of the degree of**

**Bachelor of Technology
in
Information Technology**

by

M.SREERAM

21N31A1291

K.SIDDHARTHA

21N31A1276

G.PAVAN KUMAR

22N35A1212

Under the esteemed guidance of

Mr K.N.KOUSHIL REDDY
Assistant Professor



Department of Information Technology

Malla Reddy College of Engineering & Technology

(Autonomous Institution- UGC, Govt. of India)

(Affiliated to JNTUH, Hyderabad, Approved by AICTE, NBA & NAAC with 'A' Grade)

Maisammaguda, Kompally, Dhulapally, Secunderabad – 500100

website: www.mrcet.ac.in

2021-2025



Malla Reddy College of Engineering & Technology

(Autonomous Institution- UGC, Govt. of India)

(Affiliated to JNTUH, Hyderabad, Approved by AICTE, NBA & NAAC with 'A' Grade)

Maisammaguda, Kompally, Dhulapally, Secunderabad – 500100

website: www.mrcet.ac.in

CERTIFICATE

This is to certify that this is the Bonafide record of the Major Project entitled **“MISSING CHILD IDENTIFICATION USING CONVOLUTIONAL NEURAL NETWORK”**, submitted by M. SREERAM (21N31A1291), K.SIDDHARTHA (21N31A1276) and G.PAVAN KUMAR (22N35A1212) of B. Tech in the partial fulfillment of the requirements for the degree of Bachelor of Technology in Information Technology during the year 2024-2025. The results embodied in this major project report have not been submitted to any other university or institute for the award of any degree or diploma.

Internal Guide

**Mr
K.N.KOUSHIL
REDDY**

Assistant Professor

Head of the Department

**Dr.G. Sharada
Professor**

External Examiner

DECLARATION

We hereby declare that the major project titled **“MISSING CHILD IDENTIFICATION USING CONVOLUTIONAL NEURAL NETWORK”** submitted to Malla Reddy College of Engineering and Technology (UGC Autonomous), affiliated to Jawaharlal Nehru Technological University Hyderabad (JNTUH) for the award of the degree of Bachelor of Technology in Information Technology is a result of original work carried out in this project. It is further declared that the mini project report or any part thereof has not been previously submitted to any University or Institute for the award of degree or diploma.

M.SREERAM-21N31A1291

K.SIDDHARTHA-21N31A1276

G.PAVAN KUMAR-22N35A1212

ACKNOWLEDGEMENT

We feel honored to place our warm salutation to our college Malla Reddy College of Engineering and Technology (UGC-Autonomous) for giving us an opportunity to do this Project as part of our B.Tech Program. We are ever grateful to our Director **Dr. VSK Reddy** and Principal **Dr.S.Srinivasa Rao** who enabled us to have experience in engineering and gain profound technical knowledge.

We express our heartiest thanks to our HOD, **Dr. G. Sharada**, for encouraging us throughout our course and aiding in our full potential.

We would like to thank our Project Guide **Mr.K.N.Koushil Reddy** for his regular guidance, suggestions and constant encouragement. We are extremely grateful to our Project Coordinator **Mrs.N.Prameela** for her continuous monitoring and unflinching co-operation throughout project work.

We would like to thank our Class Incharge **Mrs.N.Prameela** who, in spite of being busy with her academic duties, took time to guide and keep us on the correct path.

We would also like to thank all the faculty members and supporting staff of the Department of IT and all other departments who have been helpful directly or indirectly in making our project a success.

We are extremely grateful to our parents for their blessings and prayers for the completion of our project that gave us strength to do our project.

With regards and gratitude

M.SREERAM-21N31A1291
K.SIDDHARTHA-21N31A1276
G.PAVAN KUMAR-22N35A1212

ABSTRACT

This project presents the development of a web-based Missing Child Identification Portal utilizing Convolutional Neural Networks (CNN) for face recognition. The primary objective of this portal is to assist in locating missing children by allowing users to register missing cases, upload images, and compare them with a database of known missing children. The system is built using a combination of technologies including Node.js for the backend, MySQL for the database, and Python's DeepFace library for the face recognition functionality. Upon registering a new case, an email notification system is triggered, sending relevant information and the child's photo to all registered users of the portal. The face recognition module utilizes deep learning models to match uploaded images with the existing database, providing the ability to identify a missing child. The project emphasizes the real time identification of children based on facial features while ensuring the accuracy and efficiency of the matching process. This system can significantly reduce the time required for child identification and improve the chances of reuniting missing children with their families.

TABLE OF CONTENTS

Chapter No.	Contents	Page No
1	Introduction	1
1.1	Problem Definition	2
1.2	Existing System	2
1.3	Proposed System	3
1.4	Literature Review	4
2	System Requirements	5
2.1	Hardware & Software Requirements	5
2.2	Software Requirements Specifications (SRS)	5
3	System Design	7
3.1	Modules of System	8
3.2	UML Diagrams	9
4	Implementation	14
4.1	Sample Code	14
5	Results	26
5.1	Output Screens	37
6	Conclusion	37
	References	38

1. INTRODUCTION

The Missing Child Identification Portal is a web-based solution aimed at addressing the critical issue of missing children by utilizing facial recognition technology. The project offers a user-friendly platform where parents, guardians, and concerned individuals can register and report cases of missing children. The portal includes a comprehensive user authentication system, allowing users to create accounts, log in, and access various functionalities such as registering missing child cases, updating information, and using the search function to identify missing children through facial recognition. The core of the system is built on a Convolutional Neural Network (CNN)-based face recognition model, integrated through Python's DeepFace library. When a user uploads a child's image, the system compares it against a database of registered missing children, using sophisticated face matching algorithms to identify potential matches. This technology enhances the chances of reuniting missing children with their families, even when there are changes in the child's appearance. A key feature of the system is the automated email notification service, which sends an alert to all registered users whenever a new missing child case is reported. These alerts contain the child's details, including name, age, distinguishing features, and contact information, along with a photo. This widespread dissemination of information increases public awareness and helps speed up the recovery process. The portal also includes advanced security measures to protect sensitive data, ensuring that information such as user credentials and child photos are stored securely. The backend is powered by Node.js with Express.js, and MySQL is used for the database, making the system robust, efficient, and scalable. Future improvements could include enhanced matching algorithms, mobile integration, and expanded search capabilities, making the portal an indispensable tool for locating missing children.

1.1 Problem Definition

The process of identifying a missing person often involves comparing available information about a person (such as physical characteristics, clothing, and last known location) with records from various sources (such as social media, surveillance footage, and missing persons files). This process requires extensive and complex data analysis, which is difficult and time consuming. The main purpose of our work is to create an application that helps to improve the process of finding missing persons. We focus on helping users and persons suffering from missing persons, those who may or may not be close to them. The main purpose of this study is to use Haar Cascade, an uncomplicated real-time algorithm. The purpose of this algorithm is to identify objects, but in our case, we encounter.

1.2 Existing System

In the current landscape, various platforms, both governmental and non-governmental, address the issue of missing children. These systems often rely on manual reporting, where parents and guardians provide information about the missing child via physical forms or basic web portals. Some platforms use static databases of registered missing children, allowing users to search by criteria such as age, location, or description. However, these methods are limited in their ability to accurately identify children, particularly when their physical appearance has changed over time. Additionally, these systems largely depend on human effort to circulate missing person reports and lack integration with advanced technologies like facial recognition to enhance the identification process.

Disadvantages of exiting method

Manual Processes: Existing systems largely depend on manual data entry and search, which is time-consuming and prone to errors.

Inefficient Matching: Without automated face recognition, identification relies heavily on basic information like descriptions, which may not be accurate as time progresses.

Delayed Information Distribution: The process of circulating missing child information to the public is slow, often reliant on traditional media outlets or manual sharing, limiting the speed of search and rescue operations.

Lack of Public Involvement: Current systems do not effectively engage the broader community in the search for missing children, missing an opportunity to leverage collective effort.

Security Concerns: Many systems lack robust security, making sensitive data, such as the personal information of missing children and their families, vulnerable to breaches.

1.3 Proposed System

The Missing Child Identification Portal is designed to provide a streamlined, technology-driven solution for the identification and tracking of missing children. Unlike traditional methods that rely heavily on manual intervention and human effort, our system leverages the power of Convolutional Neural Networks (CNN) and facial recognition technology to automate the matching process, significantly increasing the efficiency and accuracy of child identification.

Facial Recognition using CNN:

At the core of the proposed system is the integration of DeepFace, a CNN-based face recognition library. This allows users to upload photos of missing or found children, which are then compared against a database of registered missing children. The CNN model extracts facial features from images, making it possible to recognize faces even with slight variations, such as changes in age, expression, or angle. By using CNN, the system can handle complex image data, making it a reliable tool for identifying children even after they have been missing for extended periods.

Automated Notification System:

Upon the registration of a new missing child case, the system sends out automated emails to all registered users. This feature ensures that the public is quickly made aware of new cases, thereby increasing the chances of finding the child. This proactive approach leverages mass communication to increase community involvement and support.

Web-Based Portal:

The user-friendly web portal allows parents, guardians, and community members to report missing children, upload photographs, and access detailed information on missing cases. The portal also facilitates searching for children based on facial features, significantly reducing the time required to identify and locate a missing child. The platform is accessible from any device, making it easy for users to report children from anywhere.

1.4 Literature Review

Paper No.	Title	Authors	Key Contributions	Relevance to Project
1	DeepFace: Closing the Gap to Human-Level Performance in Face Verification	Y. Taigman, M. Yang, M. Ranzato, L. Wolf	Introduced DeepFace, a CNN-based system achieving near-human accuracy in face verification.	Demonstrates how CNNs can enhance facial recognition for matching missing children's faces in databases.
2	FaceNet: A Unified Embedding for Face Recognition and Clustering	Florian Schroff, Dmitry Kalenichenko, James Philbin	Proposed mapping face images into a Euclidean space where distance indicates similarity.	Forms the foundation for facial matching, even with changes due to aging or other factors.
3	Facial Recognition Technology: Security vs. Privacy in the Modern Age	R.P.Mahendru, S. K. Gupta	Discusses ethical challenges, privacy concerns, and data security in facial recognition.	Ensures the project adheres to ethical guidelines when handling sensitive missing children's data.
4	Automated Face Recognition for Surveillance: Challenges and Solutions	A. Hadid, M. Pietikäinen, T. Ahonen	Examines challenges like image quality, facial expressions, and occlusions in surveillance.	Helps design a robust system that works with diverse images from parents/law enforcement.

2. SYSTEM REQUIREMENTS

2.1.1 Hardware Requirements

1. Processor: Intel Core i5 or higher (or equivalent AMD processor)
2. RAM: 8 GB or more (for handling the face recognition processing efficiently)
3. Hard Disk: 250 GB of storage space (for storing images and database records)
4. Graphics Card: Basic GPU support (NVIDIA or AMD) is recommended for faster image processing using CNN models.
5. Network: A stable internet connection for sending email notifications and accessing web services.

2.1.2 Software Requirements

1. Operating System:
 - Linux-based server operating system for hosting the web application.
 - Compatibility with major operating systems for end-users (Windows, macOS, Linux).
 - We have used Windows 11 device.
2. Web Server:
 - Deployment of a web server (e.g., Apache, Nginx) to serve the web application.
 - Configuration for secure data transmission (HTTPS).
3. Database Management System:
 - Database management system (e.g., PostgreSQL, MySQL) for storing image data.
 - Efficient indexing and query optimization for quick data retrieval.
 - We have used SQLITE database for storing our data.
4. Programming Language:
 - Python as the core programming language for system development.
5. Web Framework:
 - Django web framework for building the web application.
 - Django's ORM for database interactions
6. Machine Learning Libraries:
 - Integration of machine learning libraries such as face-recognition and OpenCV for facial recognition.
 - Compatibility with the selected libraries for training and inference.

7. Version Control:

- Version control system (e.g., Git) for tracking changes to the codebase.
- Our entire codebase is available in GitHub for learning and collaborative purposes.

8. Development Tools:

- Integrated Development Environment (IDE) for coding and debugging (e.g., Visual Studio Code, PyCharm)

9. Dependency Management:

- Utilization of package management tools (e.g., pip) for handling Python dependencies.

10. Email Services:

- Integration with email services for sending alerts (e.g., SMTP) we have used GMAIL SMPT tool for demonstration purposes.

11. Future Integration Tools (if applicable):

- Tools and libraries for video analysis integration, mobile application development, and collaboration with law enforcement databases.

2.3 Software Requirements Specification (SRS)

2.3.1 Functional Requirements

These define the specific functionalities the system must perform.

1. User Authentication

- Users should be able to create accounts with unique usernames and passwords.
- Different user roles (e.g., family members, police) with varying levels of access should be implemented.

2. Image Upload Module

- Users must be able to upload images of missing persons with accompanying information.
- Supported image formats should be clearly defined (e.g., JPEG, PNG).
- There should be restrictions on image size to optimize system performance

3. Face Recognition

- The system should employ machine learning algorithms for facial recognition.
- Extracted facial features should be compared against stored images for potential matches.
- Face encodings should be used for accurate matching.
- The system must provide a confidence score for each match.

4. Database Management

- The system should store and manage a database of images along with associated information.
- Efficient indexing and retrieval mechanisms should be implemented for quick comparison during face recognition.

5. Alert Generation

- When a match is found, the system should generate alerts.
- Alerts should include details about the missing person's location and relevant information.
- Notification methods, such as email, should be configurable.

2.3.2 Non-Functional Requirements

These define the system's quality attributes and constraints.

1. Performance:

- The system is capable of handling a large number of image uploads and comparisons efficiently.
- Response times for face recognition and database queries meets acceptable standards.

2. Security:

- User data, especially images and personal information, are securely stored and transmitted.
- Access controls are to be in place to prevent unauthorized access to sensitive information.

3. Scalability:

- The system is designed to scale easily to accommodate a growing database and user base.

4. Reliability:

- The system is reliable, with minimal downtime for maintenance or upgrades.

5. Usability:

- The user interface is intuitive and user-friendly.
- Clear error messages and guidance is provided to users.

6. Compatibility :

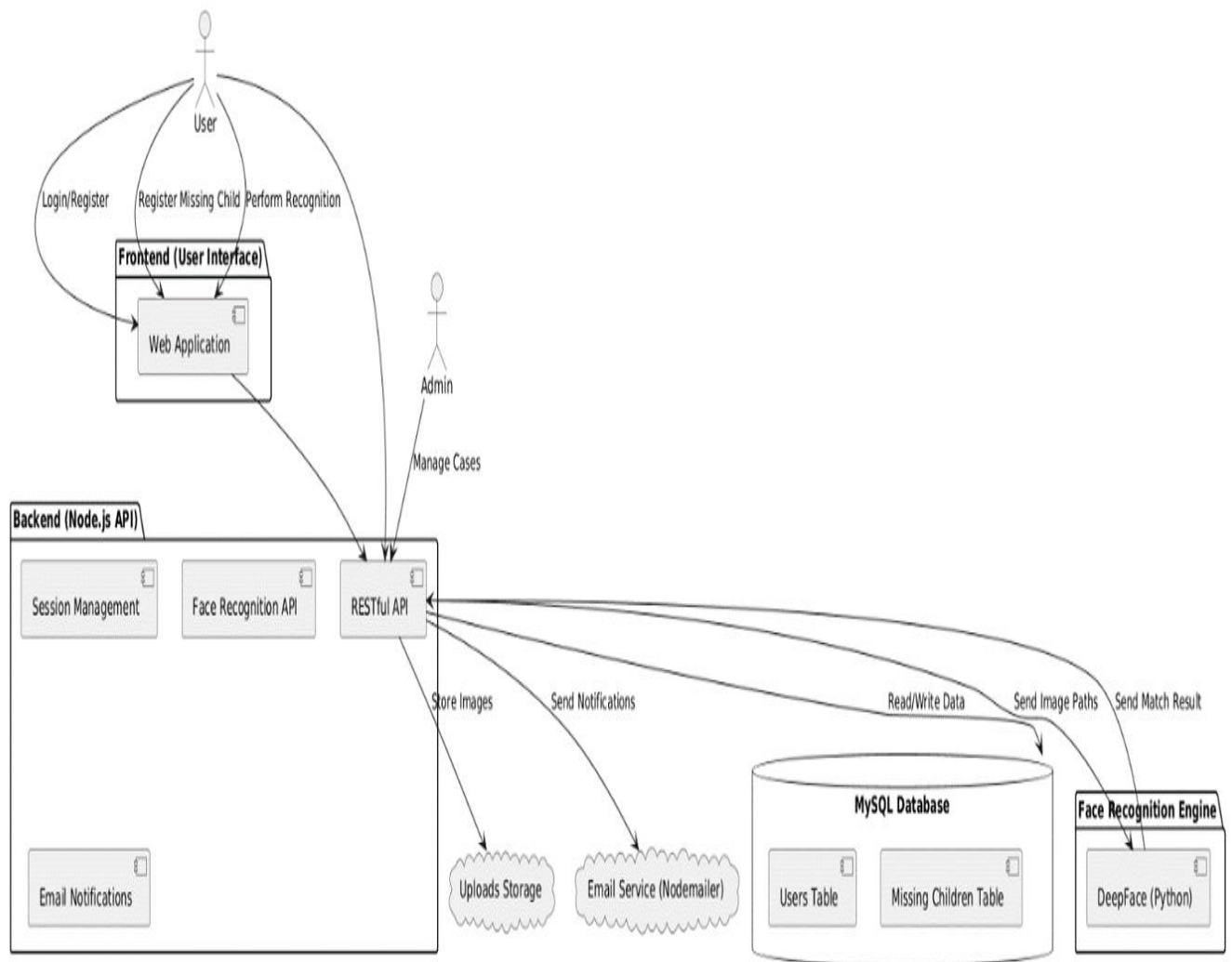
- The system should be compatible with commonly used web browsers and devices.
- Mobile applications, if developed, should support major operating systems.

7. Future Expandability:

- The system architecture should be designed to facilitate easy integration of future enhancements.
- Such as improved facial recognition algorithms and real-time video analysis.

3 SYSTEM DESIGN

System Architecture:



3.1 MODULES OF THE SYSTEM

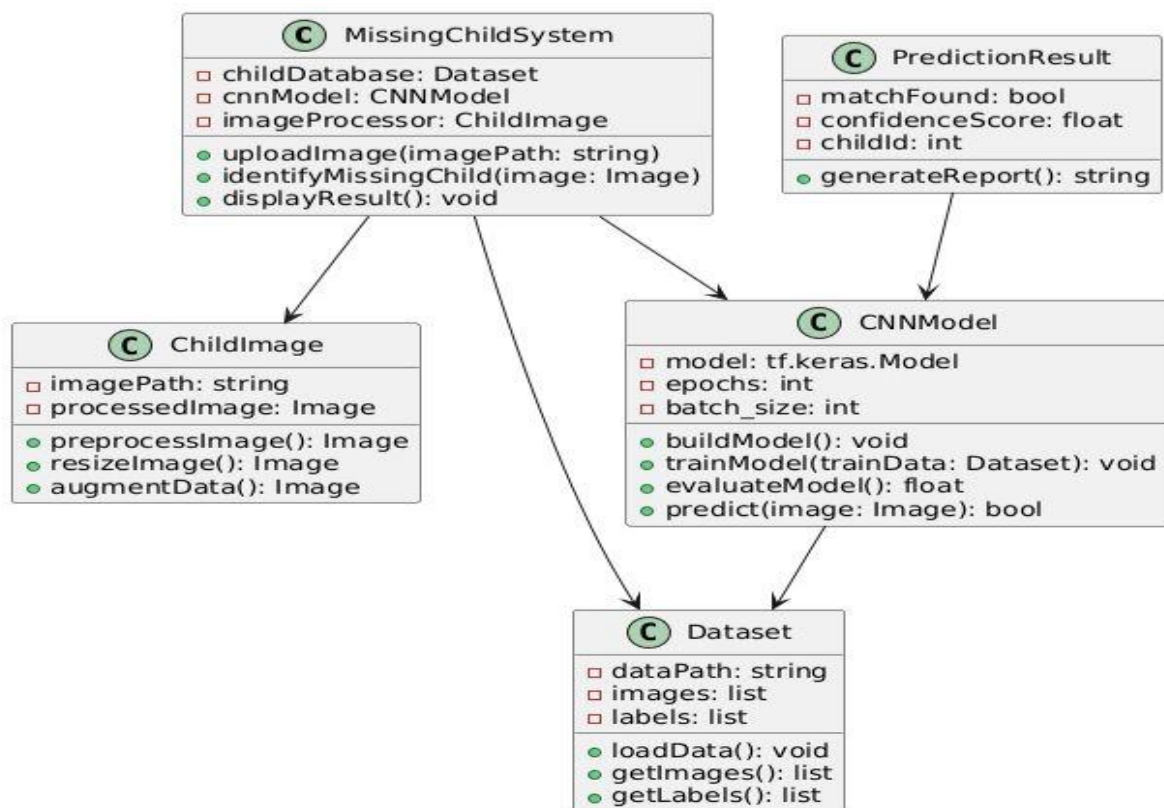
1. Input Image / Register Case:
Users initiate the system by providing an input image or registering a case. This can be done through the system's user interface or an API endpoint.
2. Get Registered:
The system processes the registration, storing relevant details in the database. This includes user information, case details, and a reference to the provided image.
3. Check for Face Detection:
Utilizing OpenCV for facial recognition, the system checks if a face is detected in the registered image. This step involves preprocessing the image, extracting facial features, and utilizing the face recognition library to identify faces.
4. Send Mail and Alerts (On Face Detection):
Description: If a face is successfully detected, the system triggers actions such as sending email notifications and alerts to notify relevant parties. This could include case investigators, administrators, or other designated individuals.
5. Search Again (On No Face Detection):
In the absence of face detection, the system prompts the user to search again. This iterative process continues until a valid face is detected or the user decides to terminate the search.
6. Admin Can Edit, Modify, Delete Information:
Administrative users have the capability to access, edit, modify, or delete information stored in the system. This functionality is accessible through a secure admin interface, ensuring control and management of the database records.

3.2 UML DIAGRAMS

UML is a method for describing the system architecture in detail using the blueprint. UML represents a collection of the best engineering practice that has proven successful in the modeling of large and complex systems. UML is a very important part of developing object-oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects. Using the helps UML helps project teams communicate to explore potential designs and validate the architectural design of the software

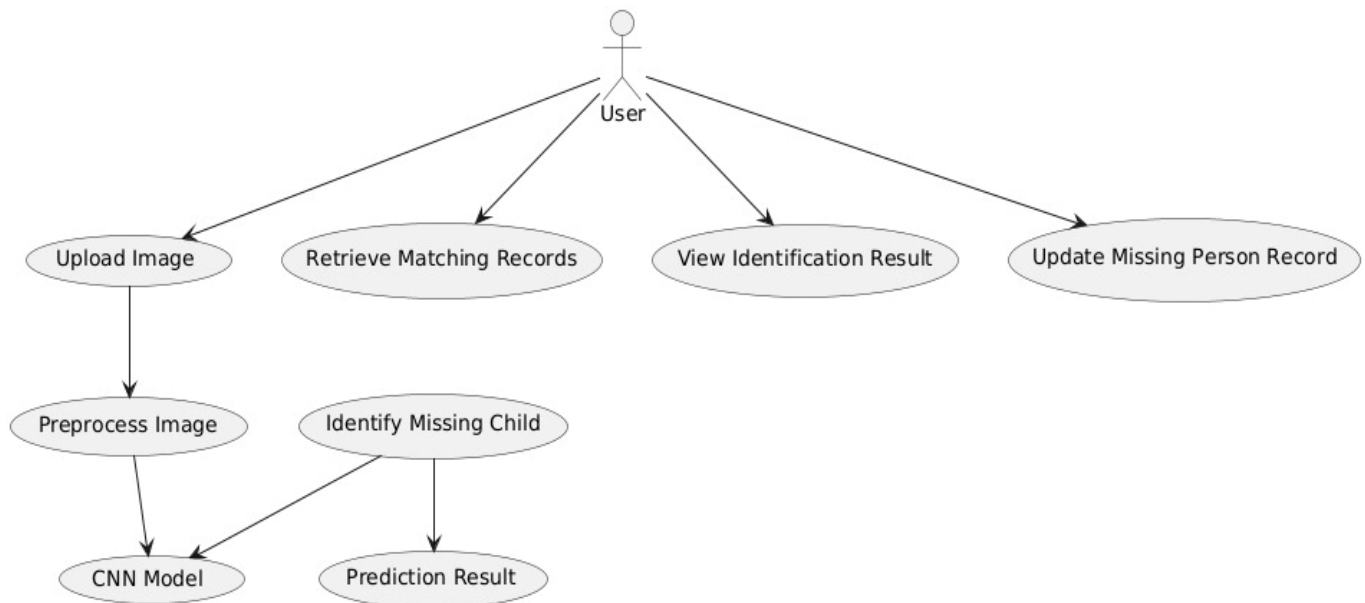
CLASS DIAGRAM

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information



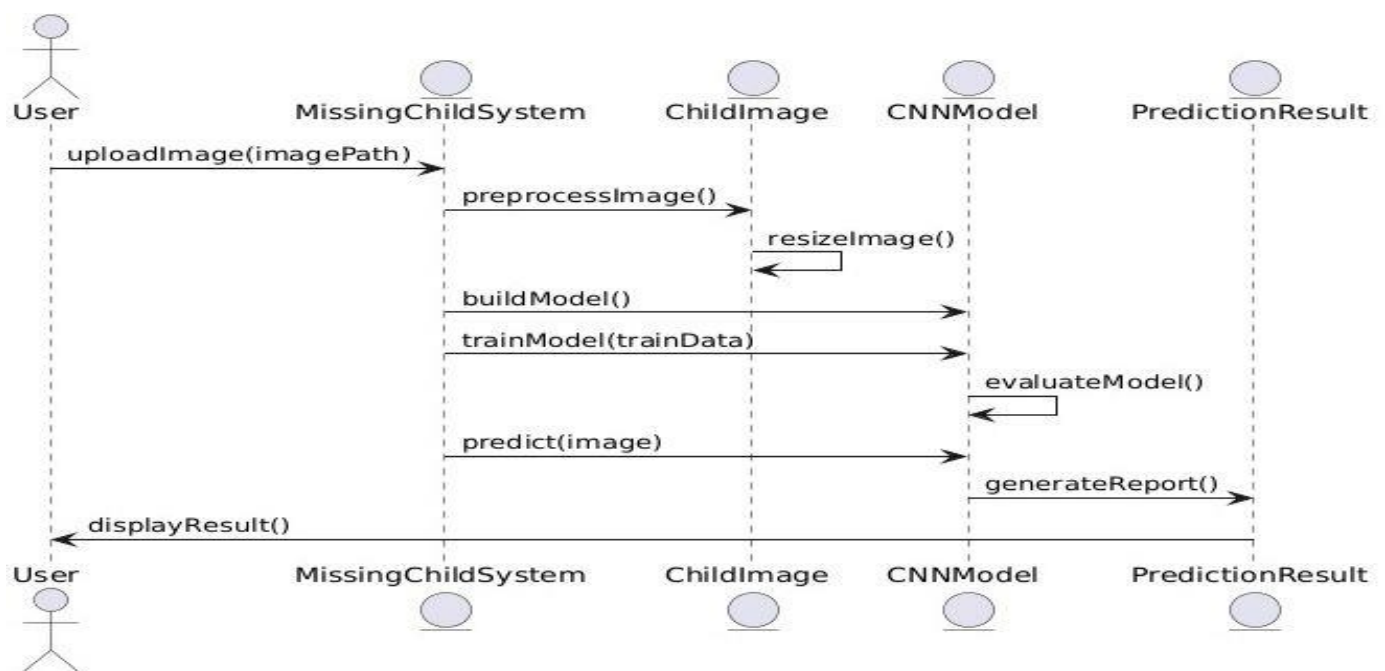
USE CASE DIAGRAM

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted



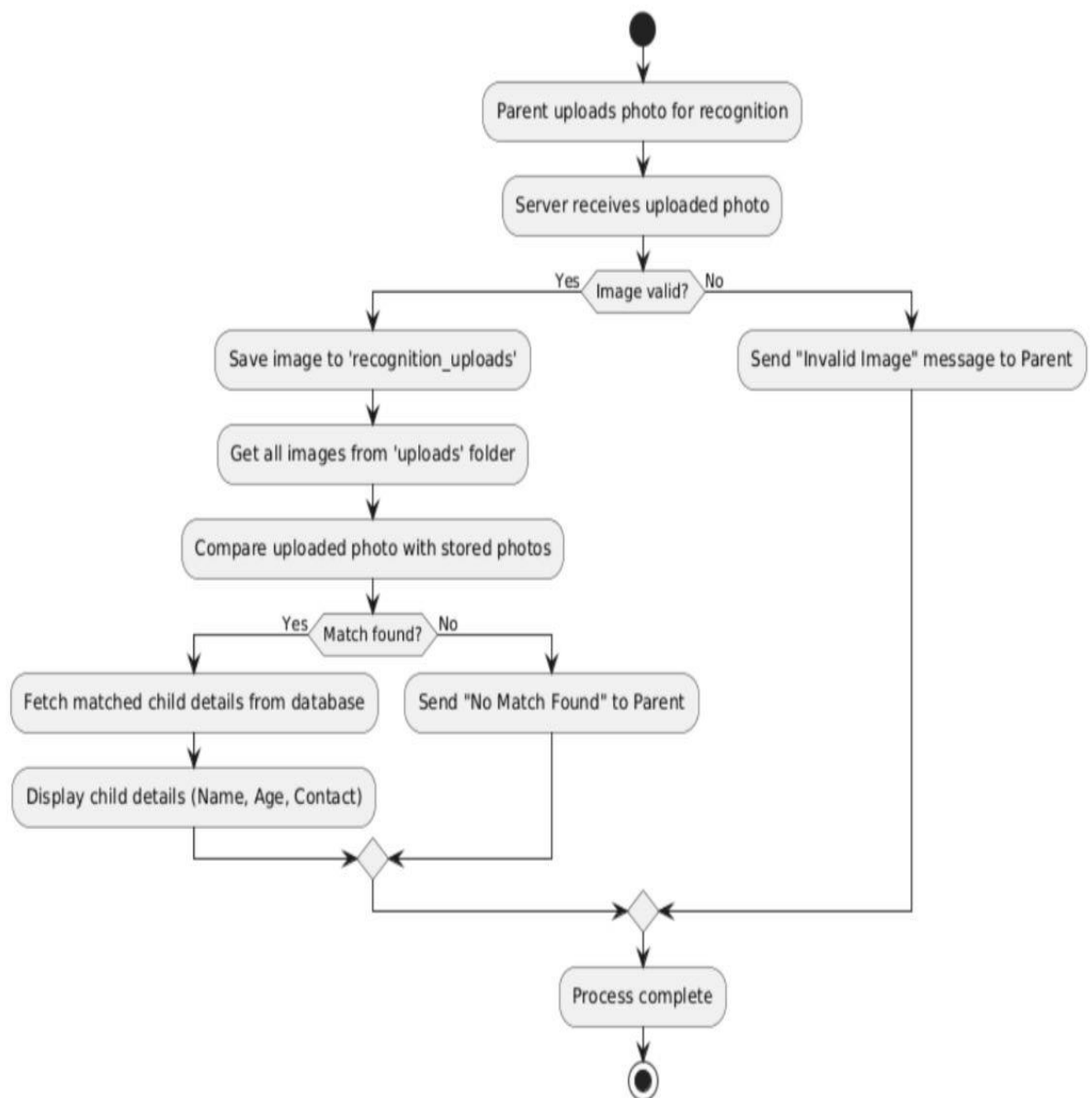
SEQUENCE DIAGRAM

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is the construction of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagram



ACTIVITY DIAGRAM

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control



4.IMPLEMENTATION

4.1 SAMPLE CODE

```
# from django.shortcuts import render,redirect
# from .models import*
# from django.contrib import messages
# from django.core.mail import send_mail
# from django.template.loader import render_to_string
# from datetime import datetime
# import face_recognition
# import cv2
# from twilio.rest import Client
# from django.shortcuts import get_object_or_404, redirect
# from django.http import HttpResponse

# #Add yourr own credentials
# account_sid = 'xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx'
# auth_token = 'xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx'
# twilio_whatsapp_number = '+14155238886'

# # Create your views here.
# def home(request):
#     return render(request,"index.html")

# def send_whatsapp_message(to,context):
#     client = Client(account_sid, auth_token)
#     whatsapp_message = (
#         f"Dear {context['fathers_name']},\n\n"
#         f"We are pleased to inform you that the missing person missing from {context['missing_from']} and
#         you were concerned about has been found. "
#         f"The person was located in a camera footage, and we have identified their whereabouts.\n\n"
#         f"Here are the details:\n"
    
```

```

# f" - Name: {context['first_name']} {context['last_name']}\n"
# f" - Date and Time of Sighting: {context['date_time']}\n"
# f" - Location: {context['location']}\n"
# f" - Aadhar Number: {context['aadhar_number']}\n\n"
# #"We understand the relief this news must bring to you. If you have any further questions or require
more information, please do not hesitate to reach out to us.\n\n"
# "Thank you for your cooperation and concern in this matter.\n\n"
# "Sincerely,\n\n"
# "Team Bharatiya Rescue ")
# message = client.messages.create(
#     body=whatsapp_message,
#     from_='whatsapp:' + twilio_whatsapp_number,
#     to='whatsapp:' + to
# )

# print(f"WhatsApp message sent: {message.sid}")

# def detect(request):
#     video_capture = cv2.VideoCapture(0)

#     # Initialize a flag to track if a face has been detected in the current video stream
#     face_detected = False

#     while True:
#         ret, frame = video_capture.read()

#         # Find face locations and encodings in the current frame
#         face_locations = face_recognition.face_locations(frame)
#         face_encodings = face_recognition.face_encodings(frame, face_locations)

#         for face_encoding, (top, right, bottom, left) in zip(face_encodings, face_locations):
#             # Compare detected face with stored face images
#             for person in MissingPerson.objects.all():

```

```

#         stored_image = face_recognition.load_image_file(person.image.path)
#         stored_face_encoding = face_recognition.face_encodings(stored_image)[0]

#         # Compare face encodings using a tolerance value
#         #tolerance = 0.6 # Adjust this tolerance as needed
#         matches = face_recognition.compare_faces([stored_face_encoding], face_encoding)

#         if any(matches):
#             name = person.first_name + " " + person.last_name
#             cv2.rectangle(frame, (left, bottom - 35), (right, bottom), (0, 0, 255), cv2.FILLED)
#             font = cv2.FONT_HERSHEY_DUPLEX
#             cv2.putText(frame, name, (left + 6, bottom - 6), font, 1.0, (255, 255, 255), 1)

#         # Check if a face has already been detected in this video stream
#         if not face_detected:
#             print("Hi " + name + " is found")

#         current_time = datetime.now().strftime('%d-%m-%Y %H:%M')
#         subject = 'Missing Person Found'
#         from_email = 'mrcet.cseb.2025@gmail.com'
#         recipientmail = person.email
#         recipient_phone_number = '+91'+str(person.phone_number)
#         print(recipient_phone_number)
#         context = {"first_name":person.first_name,"last_name":person.last_name,
#                   'fathers_name':person.father_name,"aadhar_number":person.aadhar_number,
#                   "missing_from":person.missing_from,"date_time":current_time,"location":"India"
#                 }

#         #send_wapmessage(context,current_time,wapnum)
#         send_whatapp_message(recipient_phone_number, context)
#         html_message = render_to_string('findemail.html',context = context)
#         # Send the email
#         send_mail(subject,'Hello', from_email, [recipientmail], fail_silently=False,
html_message=html_message)

```

```

#             face_detected = True # Set the flag to True to indicate a face has been detected
#             break # Break the loop once a match is found

#         # Check if no face was detected in the current frame
#         if not face_detected:
#             name = "Unknown"
#             cv2.rectangle(frame, (left, bottom - 35), (right, bottom), (0, 0, 255), cv2.FILLED)
#             font = cv2.FONT_HERSHEY_DUPLEX
#             cv2.putText(frame, name, (left + 6, bottom - 6), font, 1.0, (255, 255, 255), 1)

#         # Display the resulting image
#         cv2.imshow('Camera Feed', frame)

#         # Hit 'q' on the keyboard to quit!
#         if cv2.waitKey(1) & 0xFF == ord('q'):
#             break

#     video_capture.release()
#     cv2.destroyAllWindows()
#     return render(request, "surveillance.html")

# def surveillance(request):
#     return render(request, "surveillance.html")

# def register(request):
#     if request.method == 'POST':
#         first_name = request.POST.get('first_name')
#         last_name = request.POST.get('last_name')
#         father_name = request.POST.get('fathers_name')
#         date_of_birth = request.POST.get('dob')
#         address = request.POST.get('address')
#         phone_number = request.POST.get('phonenum')
#         aadhar_number = request.POST.get('aadhar_number')

```

```

# missing_from = request.POST.get('missing_date')
# email = request.POST.get('email')
# image = request.FILES.get('image')
# gender = request.POST.get('gender')
# aadhar = MissingPerson.objects.filter(aadhar_number=aadhar_number)
# if aadhar.exists():
#     messages.info(request, 'Aadhar Number already exists')
#     return redirect('/register')
# person = MissingPerson.objects.create(
#     first_name = first_name,
#     last_name = last_name,
#     father_name = father_name,
#     date_of_birth = date_of_birth,
#     address = address,
#     phone_number = phone_number,
#     aadhar_number = aadhar_number,
#     missing_from = missing_from,
#     email = email,
#     image = image,
#     gender = gender,
# )
# person.save()
# messages.success(request, 'Case Registered Successfully')
# current_time = datetime.now().strftime('%d-%m-%Y %H:%M')
# subject = 'Case Registered Successfully'
# from_email = 'pptodo01@gmail'
# recipientmail = person.email
# context = {"first_name":person.first_name,"last_name":person.last_name,
#            'fathers_name':person.father_name,"aadhar_number":person.aadhar_number,
#            "missing_from":person.missing_from,"date_time":current_time}
# html_message = render_to_string('regmail.html',context = context)
# # Send the email
# send_mail(subject,"", from_email, [recipientmail], fail_silently=False, html_message=html_message)

```

```

# return render(request,"register.html")

# def missing(request):
#     queryset = MissingPerson.objects.all()
#     search_query = request.GET.get('search', "")
#     if search_query:
#         queryset = queryset.filter(aadhar_number__icontains=search_query)

#     context = {'missingperson': queryset}
#     return render(request,"missing.html",context)

# def delete_person(request, person_id):
#     person = get_object_or_404(MissingPerson, id=person_id)
#     person.delete()
#     return redirect('missing') # Redirect to the missing view after deleting

# def update_person(request, person_id):
#     person = get_object_or_404(MissingPerson, id=person_id)

#     if request.method == 'POST':
#         # Retrieve data from the form
#         first_name = request.POST.get('first_name', person.first_name)
#         last_name = request.POST.get('last_name', person.last_name)
#         fathers_name = request.POST.get('fathers_name', person.fathers_name)
#         dob = request.POST.get('dob', person.dob)
#         address = request.POST.get('address', person.address)
#         email = request.POST.get('email', person.email)
#         phonenum = request.POST.get('phonenum', person.phonenum)
#         aadhar_number = request.POST.get('aadhar_number', person.aadhar_number)
#         missing_date = request.POST.get('missing_date', person.missing_date)
#         gender = request.POST.get('gender', person.gender)

```

```

#     # Check if a new image is provided
#     new_image = request.FILES.get('image')
#     if new_image:
#         person.image = new_image

#     # Update the person instance
#     person.first_name = first_name
#     person.last_name = last_name
#     person.fathers_name = fathers_name
#     person.dob = dob
#     person.address = address
#     person.email = email
#     person.phonenum = phonenum
#     person.aadhar_number = aadhar_number
#     person.missing_date = missing_date
#     person.gender = gender

#     # Save the changes
#     person.save()

#     return redirect('missing') # Redirect to the missing view after editing

#     return render(request, 'edit.html', {'person': person})

from django.shortcuts import render, redirect
from .models import *
from django.contrib import messages
from django.core.mail import send_mail
from django.template.loader import render_to_string
from datetime import datetime
import face_recognition
import cv2
from django.shortcuts import get_object_or_404

```

```

from django.http import HttpResponse

# Create your views here.
def home(request):
    return render(request, "index.html")

def detect(request):
    video_capture = cv2.VideoCapture(0)
    face_detected = False

    while True:
        ret, frame = video_capture.read()
        face_locations = face_recognition.face_locations(frame)
        face_encodings = face_recognition.face_encodings(frame, face_locations)

        for face_encoding, (top, right, bottom, left) in zip(face_encodings, face_locations):
            for person in MissingPerson.objects.all():
                stored_image = face_recognition.load_image_file(person.image.path)
                stored_face_encoding = face_recognition.face_encodings(stored_image)[0]
                matches = face_recognition.compare_faces([stored_face_encoding], face_encoding)

                if any(matches):
                    name = f"{person.first_name} {person.last_name}"
                    cv2.rectangle(frame, (left, bottom - 35), (right, bottom), (0, 0, 255), cv2.FILLED)
                    font = cv2.FONT_HERSHEY_DUPLEX
                    cv2.putText(frame, name, (left + 6, bottom - 6), font, 1.0, (255, 255, 255), 1)

            if not face_detected:
                print(f"Hi {name} is found")
                current_time = datetime.now().strftime('%d-%m-%Y %H:%M')
                subject = 'Missing Person Found'
                from_email = 'mrcet.cseb.2025@gmail.com'
                recipientmail = person.email

```

```

        context = {
            "first_name": person.first_name,
            "last_name": person.last_name,
            "fathers_name": person.father_name,
            "aadhar_number": person.aadhar_number,
            "missing_from": person.missing_from,
            "date_time": current_time,
            "location": "India"
        }
        html_message = render_to_string('findemail.html', context=context)
        send_mail(subject, 'Hello', from_email, [recipientmail], fail_silently=False,
html_message=html_message)
        face_detected = True
        break

    cv2.imshow('Camera Feed', frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

    video_capture.release()
    cv2.destroyAllWindows()
    return render(request, "surveillance.html")

def surveillance(request):
    return render(request, "surveillance.html")

def register(request):
    if request.method == 'POST':
        first_name = request.POST.get('first_name')
        last_name = request.POST.get('last_name')
        father_name = request.POST.get('fathers_name')
        date_of_birth = request.POST.get('dob')
        address = request.POST.get('address')

```

```

phone_number = request.POST.get('phonenum')
aadhar_number = request.POST.get('aadhar_number')
missing_from = request.POST.get('missing_date')
email = request.POST.get('email')
image = request.FILES.get('image')
gender = request.POST.get('gender')

aadhar = MissingPerson.objects.filter(aadhar_number=aadhar_number)
if aadhar.exists():
    messages.info(request, 'Aadhar Number already exists')
    return redirect('/register')

person = MissingPerson.objects.create(
    first_name=first_name,
    last_name=last_name,
    father_name=father_name,
    date_of_birth=date_of_birth,
    address=address,
    phone_number=phone_number,
    aadhar_number=aadhar_number,
    missing_from=missing_from,
    email=email,
    image=image,
    gender=gender,
)
person.save()
messages.success(request, 'Case Registered Successfully')
current_time = datetime.now().strftime('%d-%m-%Y %H:%M')
subject = 'Case Registered Successfully'
from_email = 'pptodo01@gmail'
recipientmail = person.email
context = {
    "first_name": person.first_name,

```

```

        "last_name": person.last_name,
        'fathers_name': person.father_name,
        "aadhar_number": person.aadhar_number,
        "missing_from": person.missing_from,
        "date_time": current_time
    }
    html_message = render_to_string('regmail.html', context=context)
    send_mail(subject, "", from_email, [recipientmail], fail_silently=False, html_message=html_message)

    return render(request, "register.html")

def missing(request):
    queryset = MissingPerson.objects.all()
    search_query = request.GET.get('search', "")
    if search_query:
        queryset = queryset.filter(aadhar_number__icontains=search_query)

    context = {'missingperson': queryset}
    return render(request, "missing.html", context)

def delete_person(request, person_id):
    person = get_object_or_404(MissingPerson, id=person_id)
    person.delete()
    return redirect('missing')

def update_person(request, person_id):
    person = get_object_or_404(MissingPerson, id=person_id)

    if request.method == 'POST':
        person.first_name = request.POST.get('first_name', person.first_name)
        person.last_name = request.POST.get('last_name', person.last_name)
        person.father_name = request.POST.get('fathers_name', person.father_name)
        person.date_of_birth = request.POST.get('dob', person.date_of_birth)

```

```
person.address = request.POST.get('address', person.address)
person.email = request.POST.get('email', person.email)
person.phone_number = request.POST.get('phonenum', person.phone_number)
person.aadhar_number = request.POST.get('aadhar_number', person.aadhar_number)
person.missing_from = request.POST.get('missing_date', person.missing_from)
person.gender = request.POST.get('gender', person.gender)

new_image = request.FILES.get('image')
if new_image:
    person.image = new_image

person.save()
return redirect('missing')

return render(request, 'edit.html', {'person': person})
```

5. RESULTS

5.1 OUTPUT SCREENS

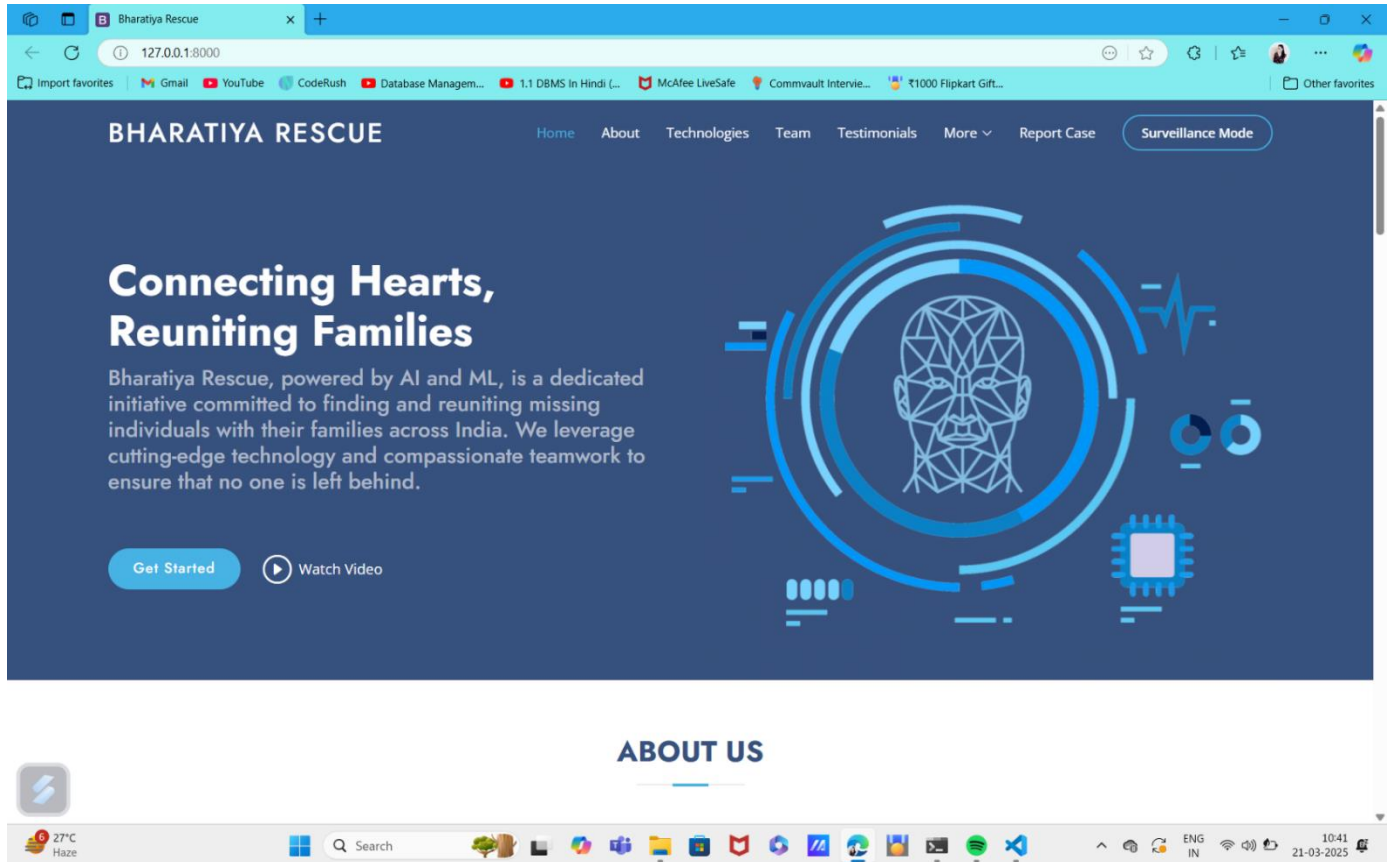


Fig. Home Page

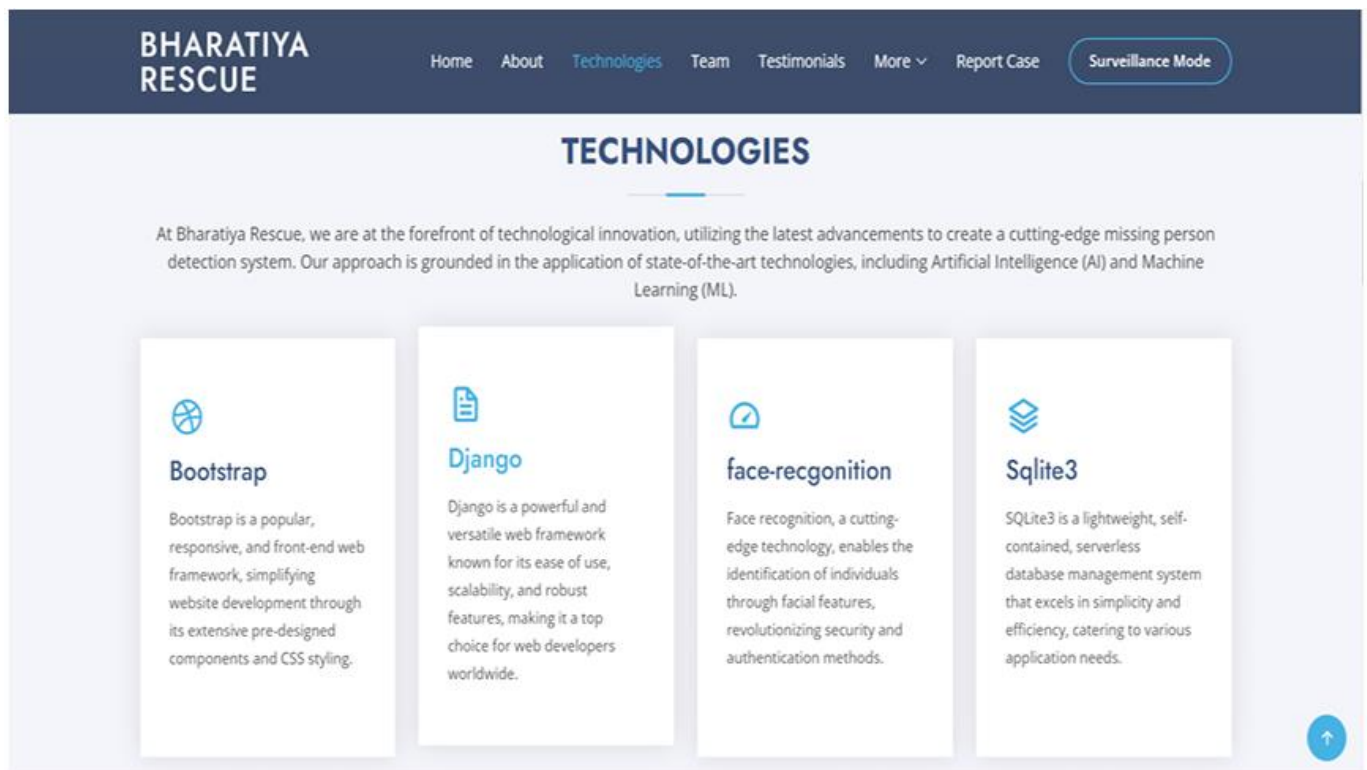


Fig. UI / UX of website

Report Missing Case

First Name:

Last Name:

Father's Name:

Date of Birth:

Address:

Email:
We'll never share your email with anyone else.

Phone Number:

Aadhar Number:

Missing Date:

Male ☐ Female ☐ Others ☐

Image: No file chosen

Fig. Registering the missing person details

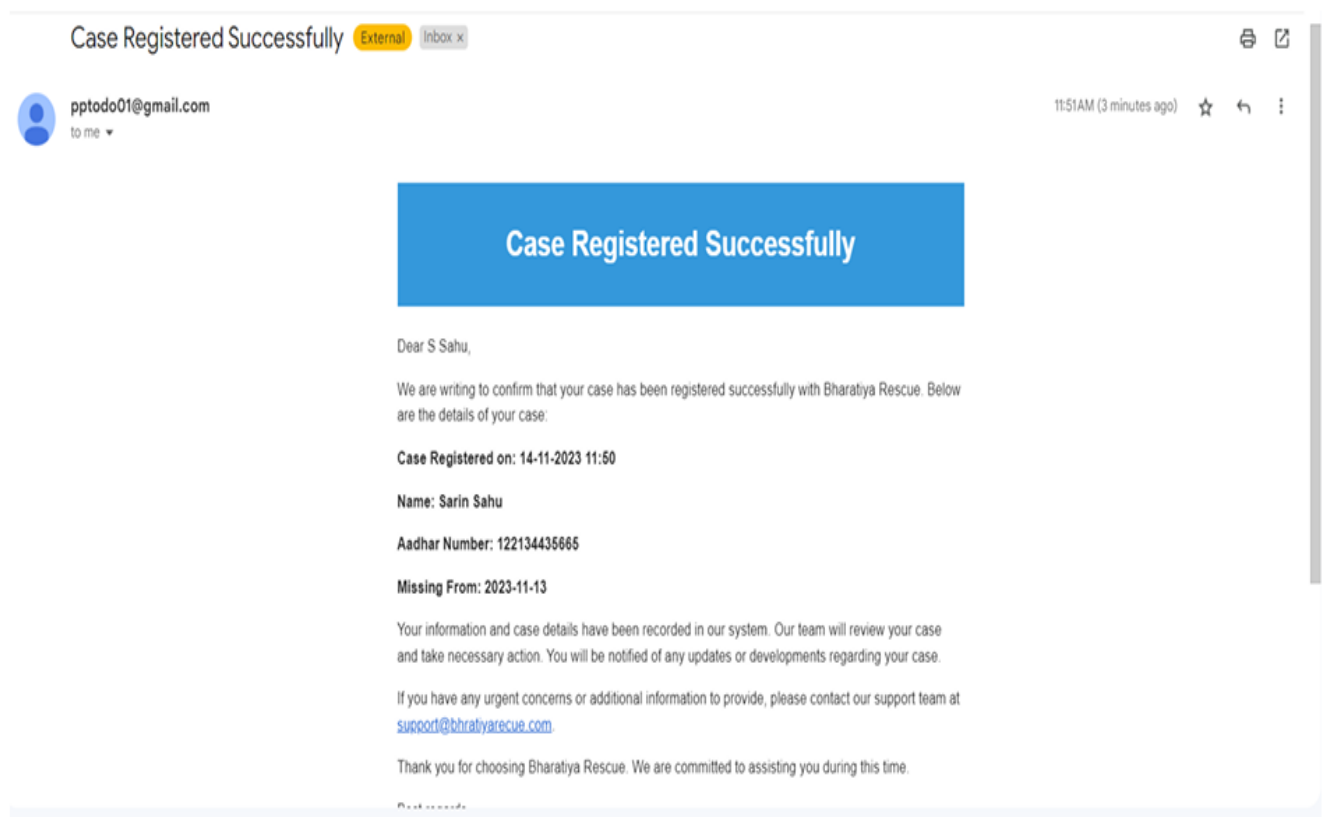


Fig. Mail sent to the registered account

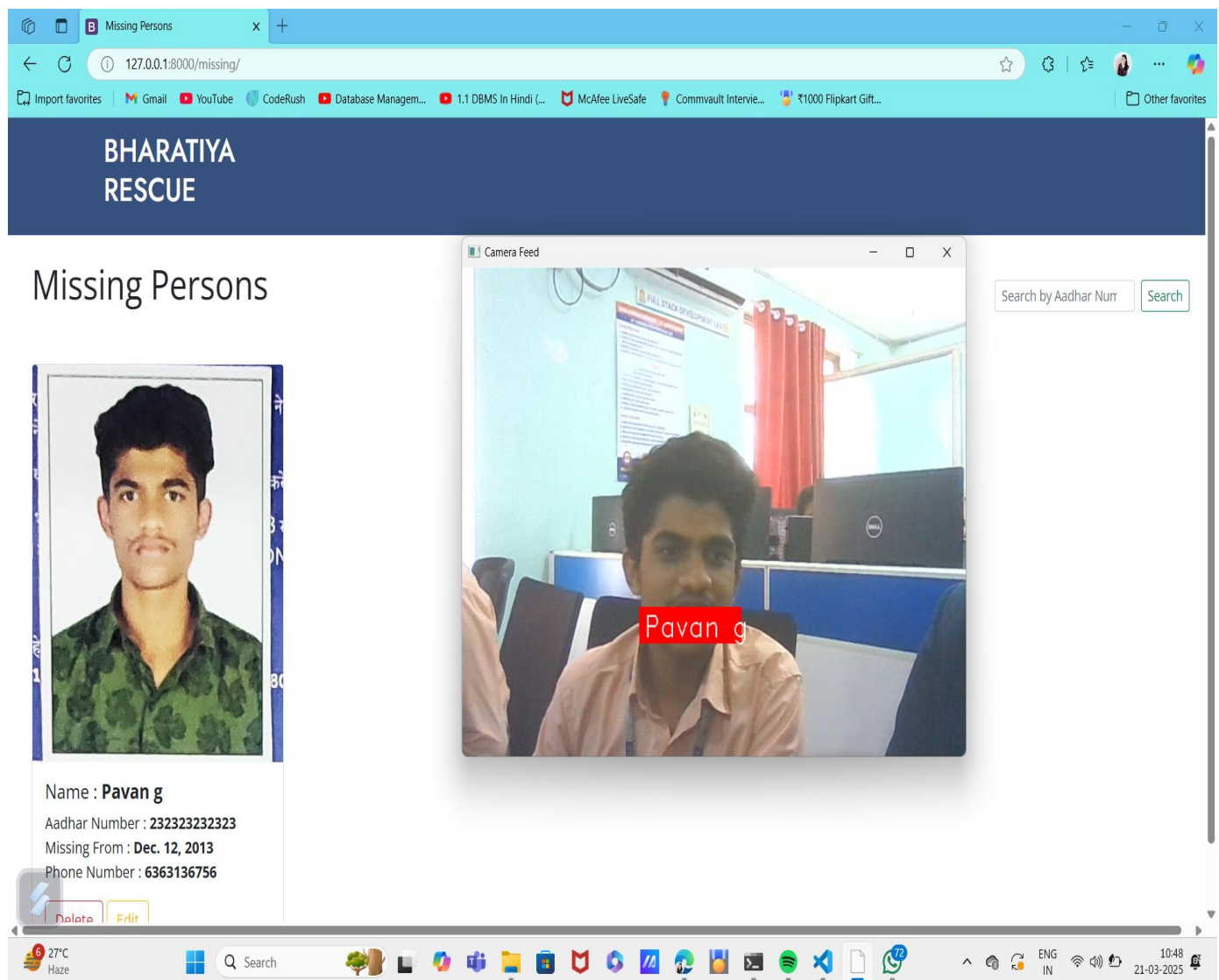


Fig. System detect the missing person

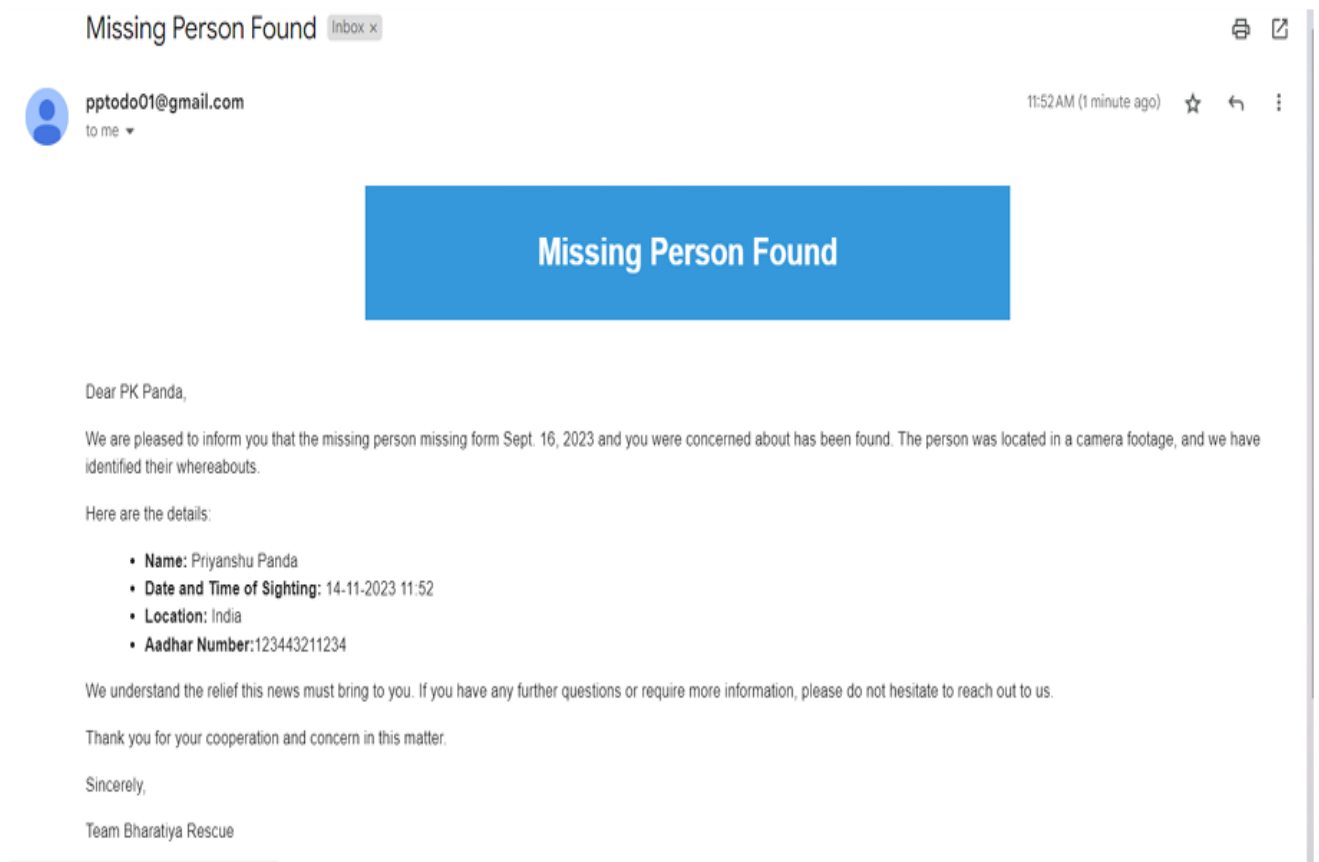


Fig. Sending alert

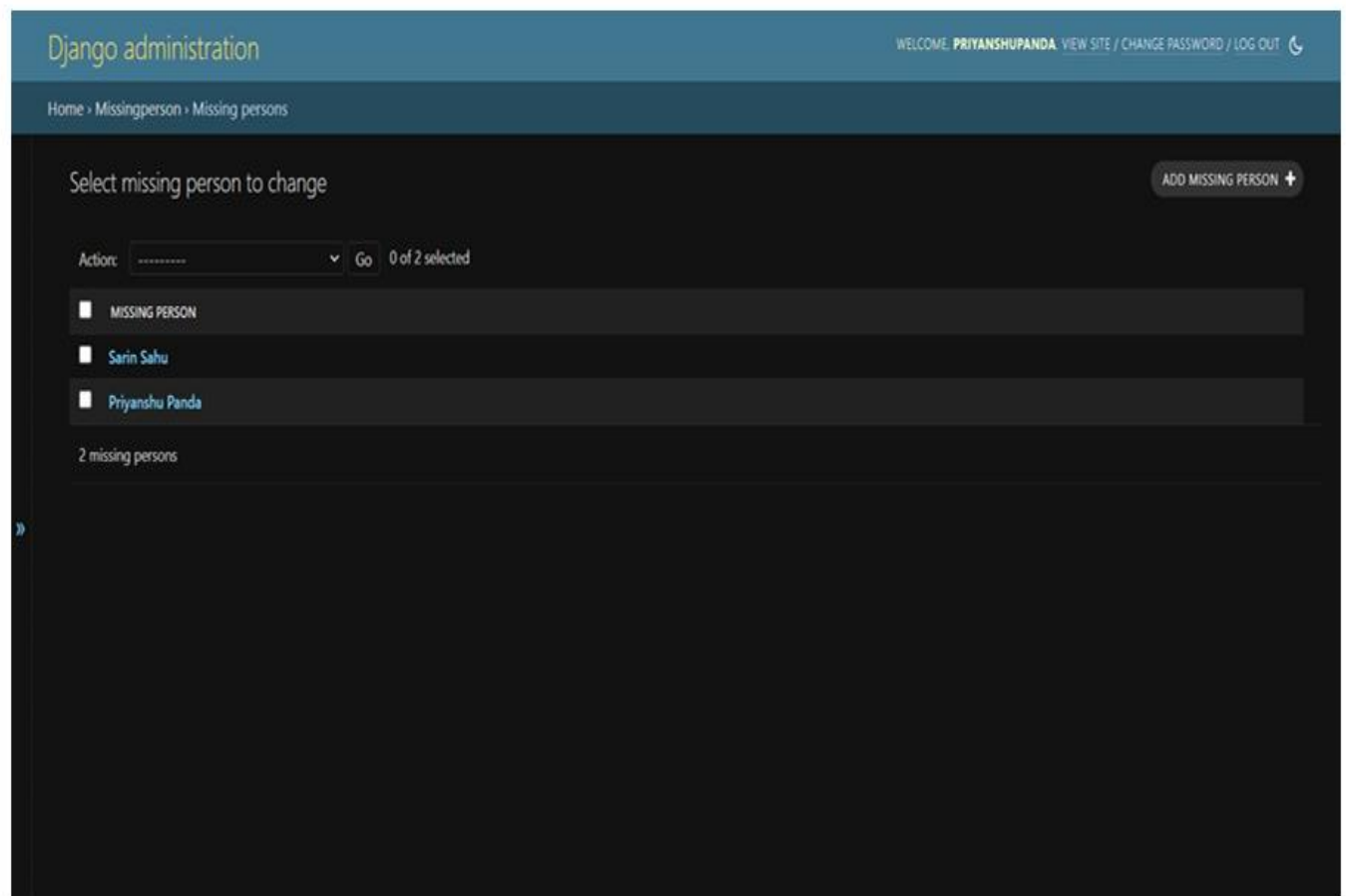


Fig. Admin's dashboard

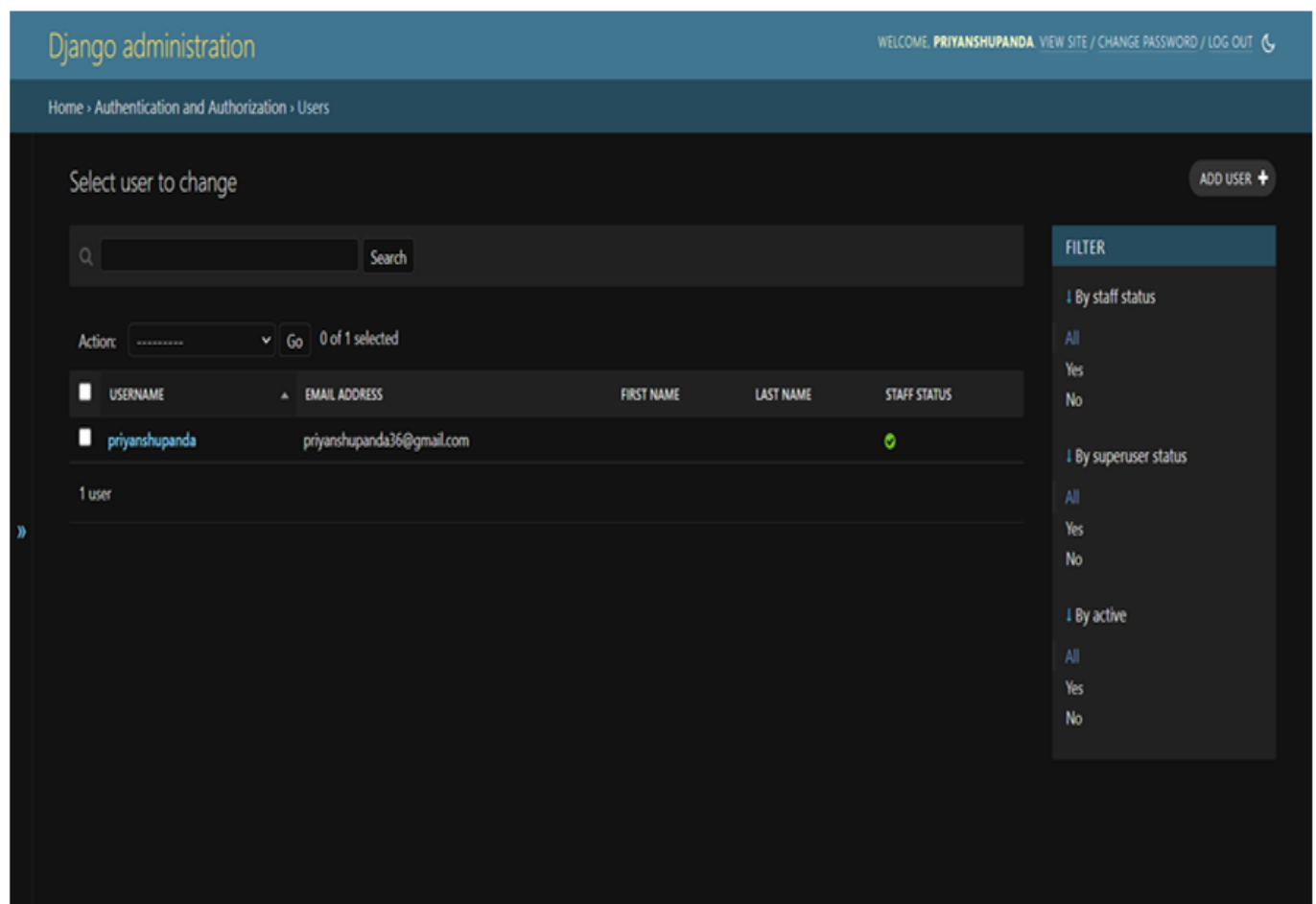


Fig. Admin Users

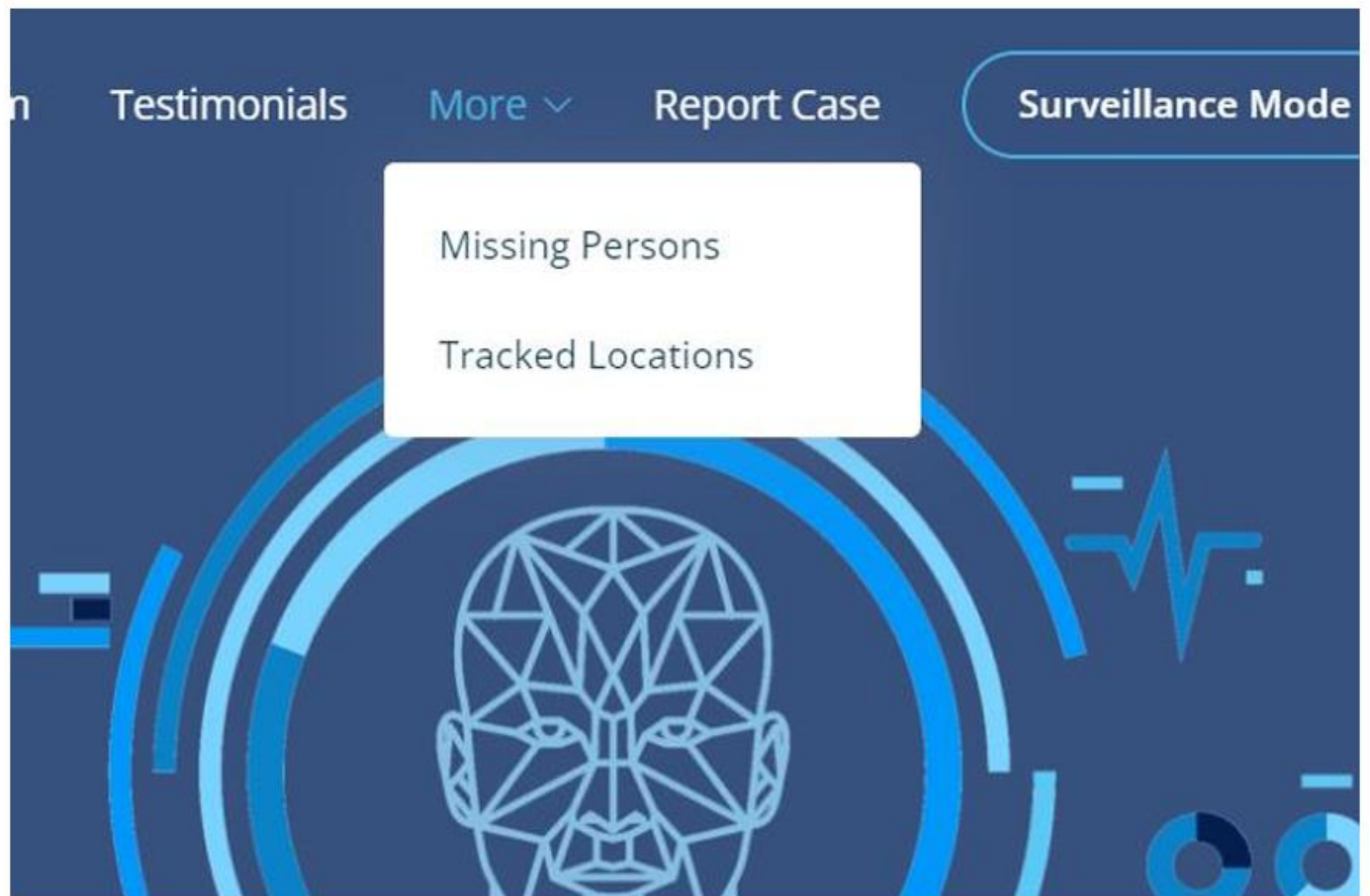


Fig. Features

Missing Persons



Name : **Priyanshu Panda**
Aadhar Number : **123443211234**
Missing From : **Sent. 16. 2023**



Name : **Sarin Sahu**
Aadhar Number : **122134435665**
Missing From : **Nov. 13, 2023**



Name : **Pratik Nayak**
Aadhar Number : **345665437890**
Missing From : **Nov. 13, 2023**
Phone Number : **8989901234**

Fig. Missing Persons

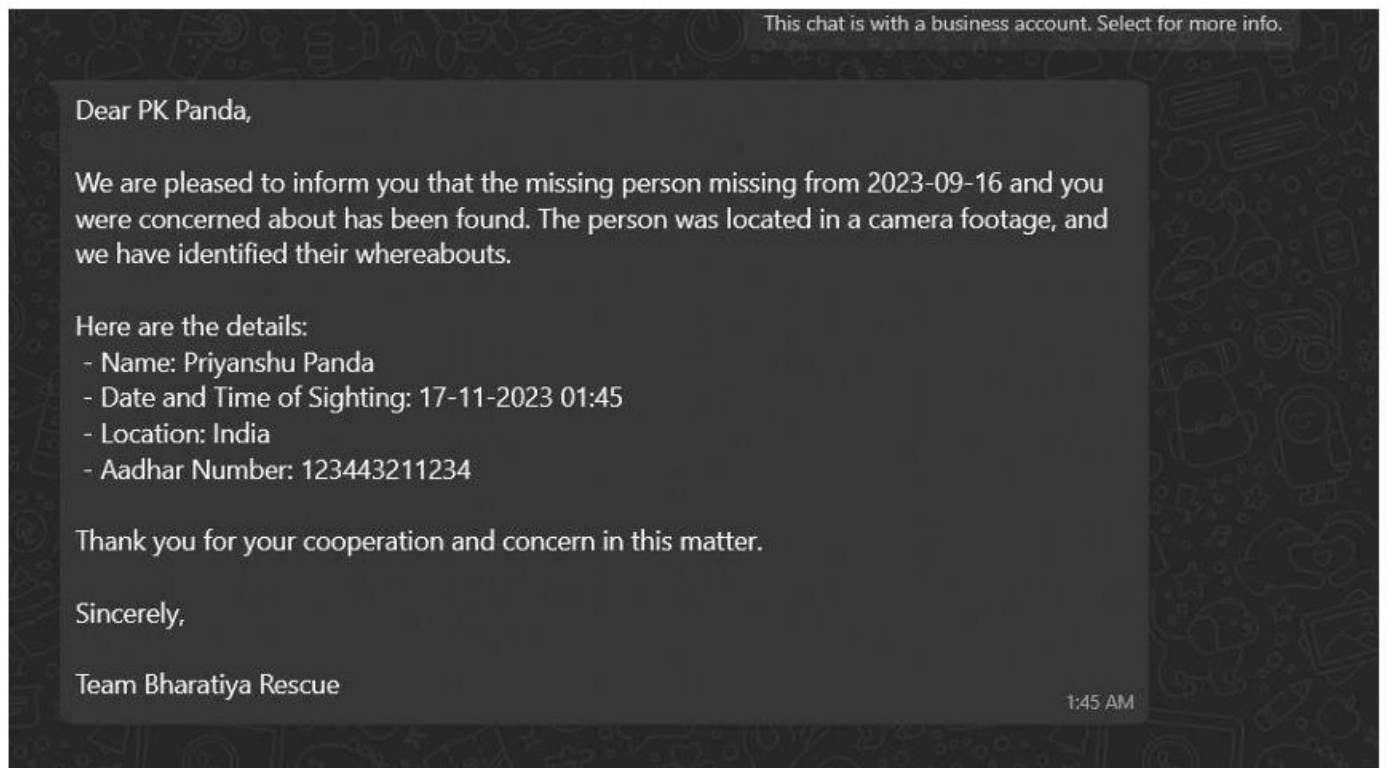


Fig. Message sent to Whats app

6. CONCLUSION

In conclusion, the "Missing Person Detection System" stands as a testament to the intersection of technology and social responsibility, providing an innovative solution to a critical societal issue. The integration of Python programming, machine learning, and web development has resulted in a powerful tool that has the potential to significantly impact the search and rescue efforts for missing individuals.. As we look ahead, the "Missing Person Detection System" holds immense potential for growth and improvement. The envisioned future scopes, including enhanced facial recognition, real-time video analysis, mobile application development, and collaboration with law enforcement, demonstrate the commitment to ongoing refinement and expansion. By continuously evolving and incorporating cutting-edge technologies, the system can further strengthen its impact in aiding authorities and families in locating missing persons. In conclusion, we are proud to have been part of a project that embodies the spirit of technological innovation for the greater good. The "Missing Person Detection System" not only showcases the capabilities of modern technology but also underscores the importance of collaboration, mentorship, and a shared vision for creating solutions that address real world challenges.

REFERENCES

1. Bootstrap:

www.bootstrap.com Source: Official website of Bootstrap, a popular open-source front-end framework.

2. ChatGPT:

Source: Developed by OpenAI, ChatGPT is a language model based on the GPT-3.5 architecture.

3. Django:

<https://www.djangoproject.com/> Source: Official website of Django, a high-level Python web framework used in the development of the "Missing Person Detection System."

4. Face Recognition Library:

<https://pypi.org/project/face-recognition/> Source: PyPI page for the face-recognition library, a crucial component in the project's facial recognition module.

5. GitHub:

<https://github.com/> Source: GitHub, a collaborative platform hosting the project's source code and facilitating version control.

6. "Missing Person Identification System" By Prof. S. S. Pawar in IJAR SCT Source:

Academic paper providing insights into missing person identification systems, contributing to the theoretical foundation of the project.

7. S. Ayyappan and S. Matilda, "Criminals and missing children identification using face recognition and web scrapping" ICSCAN 2020. Source:

IEEE conference paper detailing a related study on criminals and missing children identification, influencing the project's approach.

8. Visual Paradigm:

<https://www.visual-paradigm.com/> Source: Official website of Visual Paradigm, a tool that may have been utilized for system design and modeling.

9. YouTube:

<https://www.youtube.com/> Source: YouTube, a potential pl